

Building Without Permission in the Age of AI

Slide 1 - Building Without Permission in the Age of AI

- Open by framing the talk as part history, part present-day argument.
- Main setup: there has always been a tension between permissionless building and permissioned control.
- AI matters because it lowers the cost for ordinary people to make software, but it does not remove the need for judgement.
- If you want the sharper line early, use: permission is censorship with better branding.

Slide 2 - About me

- Early computer culture was based on sharing. If you needed software, you borrowed it, discussed it, or wrote it.
- Early personal machines often dropped you at a prompt. That matters because prompt-driven computing is not new.
- Bill Gates' 1976 letter is the early statement of proprietary software logic.
- GNU and the GPL were the pushback: software should remain a commons, not become fenced property.
- MIT licensing represents the looser branch: broad reuse is fine, but attribution stays.
- If useful, tell the Alan Cox / Linux anecdote here to make the history feel lived rather than abstract.

Slide 3 - A brief history of free and open-source

- By the 2000s, open source had effectively won the infrastructure layer.
- Linux, web servers, databases, and cryptography became standard substrate.
- Even closed products now stand on top of permissionless foundations.
- The irony is that big tech became a major maintainer of the very commons it once looked like it might replace.
- Your key phrase here is that open source won infrastructure, but permission still dominates interfaces.

Slide 4 - FOSS as the commons

- Bitcoin and Nostr are good examples because they let people build without asking anyone first.
- That changes the relationship between capital and the user base: you can build on the commons, but you cannot fully own it.
- Closed platforms lock people in and then usually enshittify.
- Open protocols create a healthier discipline: nurture the commons and the network rewards you; exploit it and the network pushes back.
- LNbits is your concrete example. It only exists because the commons existed first.
- This is where the Adam Smith invisible-hand joke lands best.

Slide 5 - The good: vibe coding

- This slide is about builder culture, not interface design.
- The point is that ordinary people are making little software projects again.
- The historical callback is the old hobbyist era: small tools, personal needs, quick experiments.
- AI makes it much easier to turn an idea into a working tool without needing a full team.
- But the scarce resource is still context, examples, and judgement.
- Important line: code is often the easy bit; judgement is the work.

Slide 6 - Doomer time...

- LLMs are very good at sounding clever.
- Once they move outside their context or training distribution, they can become very dumb very quickly.
- That is what slop really is: confidence without context.
- Add the economics point here too: these systems are still heavily subsidised and energy intensive.
- Licensing risk has not vanished either. FOSS and third-party code still come with obligations.
- At some point AI firms will need to turn a profit, and that is when the incentives harden.

Slide 7 - The bad: hype, myth, limitations

- Deliver this as observation, not management advice.
- Closed software gets weaker when users can build useful tools for themselves.
- In many products, the API becomes the new UI.
- Some systems should stay dumb, narrow, and predictable. That is often a feature, not a limitation.
- The scarce resource remains judgement, not token output.
- Keep the engineers with strong context, especially the ones who were sceptical at first. They often understand the real constraints.

Slide 8 - The ugly: AI industry is a huge bubble

- Right now, users get disproportionate capability for very little money.
- That likely means investors are subsidising the experience.
- Framed bluntly: this is a wealth transfer from VC to users, at least for now.
- Once margins matter, companies will tighten access, terms, ownership claims, and control.
- Good phrase to land here: first abundance, then dependency, then extraction.
- This is where you can mention agents becoming the next platform layer.

Slide 9 - Reality

- This is the anti-tech-bro slide.
- Engineers are not valuable because they type quickly; they are valuable because they carry structured knowledge.
- Cesar Hidalgo is useful here: knowledge is non-fungible.
- Software engineering is abstraction, tradeoffs, context, and human understanding.
- AI may remain weak for a long time outside its training set, which is another way of saying it struggles with genuinely novel work.
- Humans may remain the real context engines for much longer than the hype cycle suggests.

Slide 10 - Adjusting to current reality

- End optimistic, but not naive.
- Open source is still the substrate.
- AI is useful when it expands who gets to build.
- It becomes dangerous when it centralises power and erodes skill.
- The line to finish on is the new quote: the future belongs to people who can use powerful tools without giving up understanding.

Slide 11 - Demo

- Short concrete demo.
- Show something that makes the leverage point obvious rather than explaining every implementation detail.
- Keep it tight and let it act as evidence for the argument in the talk.

Optional spare lines / ad libs

- Permission is censorship.
- Slop is what happens when confidence outruns context.
- Closed proprietary software is weaker than it looks when the cost of building collapses.
- "I still review all the code" is the honest version of AI-assisted development.
- Sam Altman looks a bit like a GTA character who is always thinking very hard.